7.2 Структурное логирование на примере библиотеки zap

Что такое Zap?

Zap — это высокопроизводительная библиотека структурированного логирования, разработанная Uber. Она поддерживает JSON-логи, уровни логирования и гибкую настройку, что делает её подходящей для крупных систем.

Преимущества Zap

- **Скорость**: Минимизирует накладные расходы, избегая рефлексии.
- **Структурированное логирование**: JSON-логи для удобного анализа.
- **Гибкость**: Настройка уровней, форматов и направлений вывода.
- **Простота использования**: Два API — Sugared Logger и Base Logger.
- **Кастомизация**: Поддержка пользовательских полей и выборки логов.

Основное использование

```
package main

import (
   "go.uber.org/zap"
)

func main() {
   logger, _ := zap.NewProduction()
   defer logger.Sync()

   logger.Info("Информационное сообщение", zap.String("ключ", "значение"))
   logger.Error("Сообщение об ошибке", zap.Int("код", 500))
}
```

Вывод:

```
{"level":"info","ts":1622745180.123456,"msg":"Информационное сообщение","ключ":"
значение"}

{"level":"error","ts":1622745180.123457,"msg":"Сообщение об ошибке","код":500}
```

Установка

Установите Zap командой:

```
go get go.uber.org/zap
```

Импортируйте пакет:

```
import "go.uber.org/zap"
```

Преимущества Zap

1. **Высокая производительность**: Минимизация накладных расходов (Uber Zap Documentation).
2. **Структурированное логирование**: JSON-логи для анализа (SigNoz Guide).
3. **Гибкость**: Настройка уровней и форматов (Better Stack Guide).
4. **Два API**: Sugared Logger для удобства, Base Logger для производительности (Uber Zap GitHub).
5. **Кастомизация**: Пользовательские поля и выборка логов (Last9 Blog).
6. **Интеграция**: Поддержка вывода в файлы и консоль (AppMaster Blog).

Создание логгера

Zap предоставляет логгеры для разработки (`NewDevelopment`) и продакшена (`NewProduction`).

```go
package main

import (
   "go.uber.org/zap"
)

func main() {
   logger, err := zap.NewProduction()
   if err != nil {
      panic(err)
   }
   defer logger.Sync()

   devLogger, err := zap.NewDevelopment()
   if err != nil {
      panic(err)
   }
   defer devLogger.Sync()

   logger.Info("Сообщение в продакшен-логгере")
   devLogger.Info("Сообщение в разработочном логгере")
}
```

Вывод (примерный):

```
{"level":"info","ts":1622745180.123456,"msg":"Сообщение в
продакшен-логгере"}

INFO    main.go:16     Сообщение в разработочном логгере
```

Глобальный логгер

```
package main

import (
   "go.uber.org/zap"
)

func init() {
   zap.ReplaceGlobals(zap.Must(zap.NewProduction()))
}

func main() {
   zap.L().Info("Hello from Zap!")
}
```

Уровни логирования

```
package main

import (
   "go.uber.org/zap"
)

func main() {
   logger, _ := zap.NewProduction()
   defer logger.Sync()

   logger.Debug("Отладочное сообщение")
   logger.Info("Информационное сообщение")
   logger.Warn("Предупреждение") logger.Error("Ошибка")
}
```

Логирование с дополнительными полями

```go
package main

import (
   "go.uber.org/zap"
)

func main() {
   logger, _ := zap.NewProduction()
   defer logger.Sync()

   logger.Info("Пользователь вошёл в систему",
      zap.String("user_id", "12345"),
      zap.Int("attempt", 3),
      zap.Bool("success", true),
   )
}
```

Вывод (примерный):

```
{"level":"info","ts":1622745180.123456,"msg":"Пользователь вошёл в систему","user_id":"12345","attempt":3,"success":true}
```

Пользовательская конфигурация

Zap позволяет направлять логи в консоль и файлы через `OutputPaths`

```go
package main

import (
    "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
    "os"
)

func main() {
    encoderCfg := zap.NewProductionEncoderConfig()
    encoderCfg.TimeKey = "timestamp"encoderCfg.EncodeTime
    = zapcore.ISO8601TimeEncoder

    config := zap.Config{
        Level:            zap.NewAtomicLevelAt(zap.InfoLevel),
        Development:      false,
        Encoding:         "json",
        EncoderConfig:    encoderCfg,
        OutputPaths:      []string{"stdout", "app.log"},
        ErrorOutputPaths: []string{"stderr"},
        InitialFields:  map[string]interface{}{"app": "my-app"},
    }

    logger, err := config.Build()
    if err != nil {
        panic(err)
    }
    defer logger.Sync()

    logger.Info("Запуск приложения")
}
```

Вывод (примерный):

```
{"level":"info","timestamp":"2025-06-02T18:22:00.123+0300","ms
g":"Запуск приложения","app":"my-app"}
```

Выборка логов

Zap поддерживает выборку логов для уменьшения их объёма.

```
package main

import (
   "go.uber.org/zap"
   "go.uber.org/zap/zapcore"
)

func main() {
   config := zap.NewProductionConfig()
   config.Sampling = &zapcore.SamplingConfig{
      Initial:  100,
      Thereafter: 100,
   }
   logger, _ := config.Build()
   defer logger.Sync()

   for i := 0; i < 1000; i++ {
     logger.Info("Тест выборки логов")
   }
}
```

Таблица: Сравнение со стандартным логгером

Характеристика	Стандартный логгер (`log`)	Zap Logger
Производительность	Средняя	Высокая, минимизация выделений
Структурированное логирование	Нет, текстовый вывод	Да, JSON-формат
Уровни логирования	Отсутствуют	Debug, Info, Warn, Error, др.
Кастомизация	Ограниченная	Высокая, настраиваемые поля
Вывод	Только stderr или файл	Множественны е направления

Рекомендации по использованию

- Используйте `Debug` для отладки, `Info` для общей информации, `Error` для ошибок.
- Добавляйте контекст с помощью полей (`zap.String , zap.Int`).
- Настройте направления вывода для хранения и отладки.
- Используйте Sugared Logger для разработки, Base Logger для продакшена.
- Применяйте выборку логов в высоконагруженных системах.

Заключение

Zap сочетает скорость, структурированное логирование и гибкость, превосходя стандартный пакет `log` . Он идеально подходит для современных приложений, упрощая мониторинг и анализ.